Assignment 02

1. Significant earthquakes since 2150 B.C.

1.1 [5 points] Compute the total number of deaths caused by earthquakes since 2150 B.C. in each country, and then print the top 20 countries along with the total number of deaths.

➢ 思考

	Id	Year	Mo	Dy	Hr	Mn	Sec	Tsu	Vol	Country	...	Total Missing	Total Missing Description	Total Injuries	Total Injuries Description	Total Damage ($Mil)	Total Damage Description	Total Houses Destroyed	De Des
0	1	-2150	NaN	NaN	NaN	NaN	NaN	NaN	NaN	JORDAN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
1	2	-2000	NaN	NaN	NaN	NaN	NaN	1.0	NaN	SYRIA	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
2	3	-2000	NaN	NaN	NaN	NaN	NaN	NaN	NaN	TURKMENISTAN	...	NaN	NaN	NaN	NaN	NaN	1.0	NaN	
3	5877	-1610	NaN	NaN	NaN	NaN	NaN	3.0	1351.0	GREECE	...	NaN	NaN	NaN	NaN	NaN	3.0	NaN	
4	8	-1566	NaN	NaN	NaN	NaN	NaN	NaN	NaN	ISRAEL	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
5	11	-1450	NaN	NaN	NaN	NaN	NaN	NaN	NaN	ITALY	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
6	9712	-1365	NaN	NaN	NaN	NaN	NaN	4.0	NaN	SYRIA	...	NaN	NaN	NaN	NaN	NaN	3.0	NaN	
7	12	-1250	NaN	NaN	NaN	NaN	NaN	NaN	NaN	ISRAEL	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
8	13	-1050	NaN	NaN	NaN	NaN	NaN	NaN	NaN	JORDAN	...	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
9	14	-759	NaN	NaN	NaN	NaN	NaN	NaN	NaN	ISRAEL	...	NaN	NaN	NaN	NaN	NaN	3.0	NaN	

先筛选[Year]为公元前 2150 年之后的，再对[Country]进行分组，统计[Deaths]死亡人数，然后根据统计的结果得到排名前 20 的国家及死亡总人数。

➢ 代码

```
# 1.1
top20_deaths = SE[SE["Year"] >= -2150].groupby(['Country'])["Deaths"].sum().sort_values(ascending=False)

print("公元前2150年以来地震死亡人数前20国家：")
top20_deaths.head(20)
```

➢ 结果

```
公元前2150年以来地震死亡人数前20国家：

Country
CHINA          2139209.0
TURKEY         1199720.0
IRAN           1014453.0
ITALY           492236.0
SYRIA           419826.0
HAITI           323484.0
AZERBAIJAN      319251.0
JAPAN           242516.0
ARMENIA         177890.0
PAKISTAN        145085.0
IRAQ            136200.0
ECUADOR         135496.0
TURKMENISTAN    117412.0
PERU            102309.0
ISRAEL           93388.0
PORTUGAL         83572.0
GREECE           81162.0
CHILE            64277.0
INDIA            63507.0
TAIWAN           57154.0
Name: Deaths, dtype: float64
```

1.2 [10 points] Compute the total number of earthquakes with magnitude larger than 3.0 (use column Ms as the magnitude) worldwide each year, and then plot the time series. Do you observe any trend? Explain why or why not?

- 思考

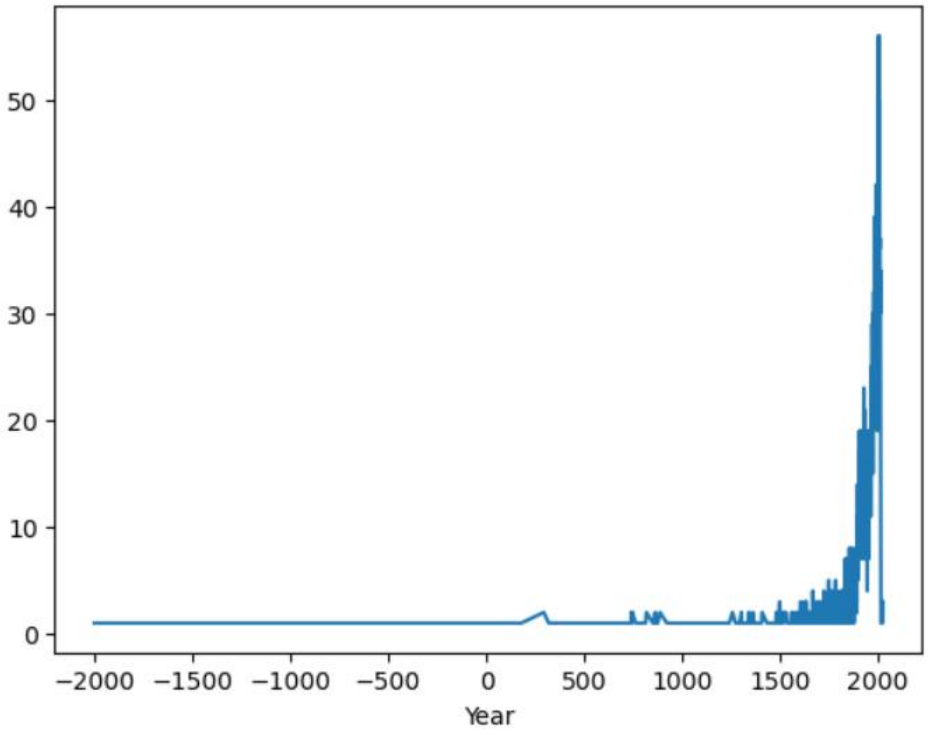
先筛选[Ms]大于 3.0，再按[Year]分组，计算地震总数，最后绘制震级大于 3.0 的时间序列图。

- 代码

```
# 1.2
over3_Ms = SE[SE["Ms"] > 3.0].groupby(['Year']).count()['Ms'].plot()

plt.show()
```

- 结果



- 结论

从图中能看到非常明显的特征，公元 1500 年之后，尤其是近代，记录到的地震数量急剧上升。但这不代表地球上的地震真的变多了，核心原因是地震观测与记录的完善程度随时间大幅提升。

1.古代（公元前到中世纪）地震记录极其不全，只有造成重大灾害的强震才会被史料记载，大量中小地震都没有留下记录，因此数值很低且平稳。

2.近现代以来，地震监测台站全球普及、监测技术进步，越来越多的中小地震能被精准记录，所以统计到的数量快速增长。

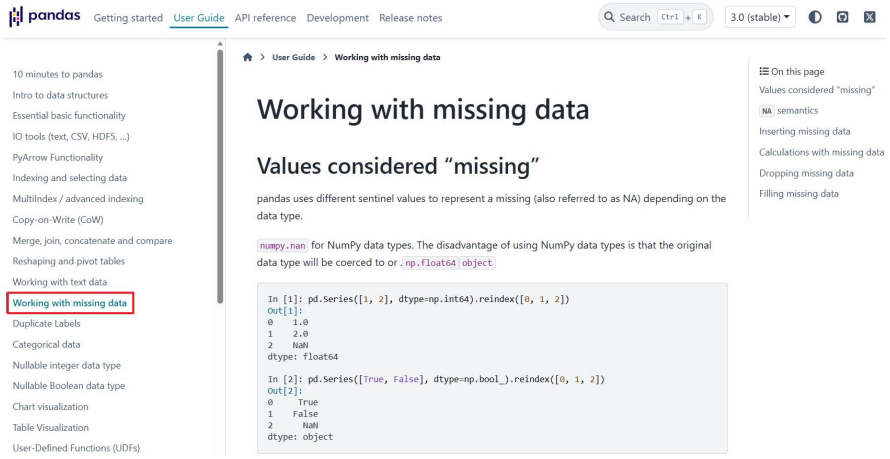
3.这是历史地震数据库的典型特征。数据反映的是被记录的地震数量，而非实际发生的地震数量。

1.3 [10 points] Write a function CountEq_LargestEq that returns (a) the total number of earthquakes since 2150 B.C. in a given country AND (b) date and location of the largest earthquake ever happened in this country. Apply CountEq_LargestEq to every country in the file, report your results in a descending order.

➢ 思考

先筛选出指定国家（形参）和[Year]为公元前 2150 年之后的地震，统计总数；再剔除震级为空的记录，找到震级最大的地震，提取它的日期和位置一起返回。

遍历所有国家调用函数，把结果整理成表格，按地震总数从高到低降序排列。



I drew inspiration from the "Working with missing data" section of the official pandas User Guide. website：Working with missing data — pandas 3.0.6 documentation

➢ 代码

```python
# 1.3
def CountEq_LargestEq(country_name):
    bf2150_ct = (SE["Country"] == country_name) & (SE["Year"] >= -2150)
    country_data = SE[bf2150_ct].copy()

    eq_count = len(country_data)

    if eq_count == 0:
        return eq_count, "无记录", "无记录"

    # ========== 先去掉震级为空的行，再找最大值 ==========
    valid_data = country_data.dropna(subset=["Ms"])
    if len(valid_data) == 0:
        return eq_count, "无有效震级记录", "无记录"
    # ================================================

    largest_eq = country_data.loc[country_data["Ms"].idxmax()]
    date_str = f"{int(largest_eq['Year'])}年"
    if pd.notnull(largest_eq["Mo"]):
        date_str += f"{int(largest_eq['Mo'])}月"
    if pd.notnull(largest_eq["Dy"]):
        date_str += f"{int(largest_eq['Dy'])}日"
    location = largest_eq["Location Name"]

    return eq_count, date_str, location
```

```python
countries = SE["Country"].unique()

result_list = []
for country in countries:
    cnt, eq_date, eq_loc = CountEq_LargestEq(country)
    result_list.append({
        "Country": country,
        "Total_Earthquakes": cnt,
        "Largest_Eq_Date": eq_date,
        "Largest_Eq_Location": eq_loc
    })

result_df = pd.DataFrame(result_list).sort_values(
    by="Total_Earthquakes",
    ascending=False
).reset_index(drop=True)

print(result_df.head(20))
```

➢ 结果

	Country	Total_Earthquakes	Largest_Eq_Date	Largest_Eq_Location
0	CHINA	626	1920年12月16日	CHINA: GANSU PROVINCE, SHANXI PROVINCE
1	JAPAN	429	869年7月13日	JAPAN: SANRIKU
2	INDONESIA	427	2004年12月26日	INDONESIA: SUMATRA: ACEH: OFF WEST COAST
3	IRAN	385	856年12月22日	IRAN: DAMGHAN, QUMIS
4	TURKEY	359	1939年12月26日	TURKEY: ERZINCAN
5	ITALY	338	1915年1月13日	ITALY: MARSICA, AVEZZANO, ABRUZZI
6	GREECE	296	365年7月21日	GREECE: CRETE: KNOSSOS
7	USA	283	1957年3月9日	ALASKA
8	PHILIPPINES	233	1897年9月21日	PHILIPPINES: MINDANAO, ZAMBOANGA, SULU, ISABELA
9	MEXICO	217	1787年3月28日	MEXICO: SAN MARCOS, OAXACA
10	CHILE	200	1730年7月8日	CHILE: VALPARAISO
11	PERU	195	1619年2月14日	PERU: TRUJILLO, PIURA, SANTA
12	RUSSIA	159	1952年11月4日	RUSSIA: KAMCHATKA PENINSULA
13	PAPUA NEW GUINEA	107	2000年11月16日	PAPUA NEW GUINEA: NEW IRELAND, DUKE OF YORK
14	TAIWAN	103	1920年6月5日	TAIWAN
15	INDIA	102	1897年6月12日	INDIA: ASSAM; BANGLADESH
16	COLOMBIA	83	1904年1月20日	PANAMA-COLOMBIA
17	AFGHANISTAN	72	1909年7月7日	AFGHANISTAN: HINDU-KUSH
18	NEW ZEALAND	72	1826年	NEW ZEALAND: FJORDLAND
19	ECUADOR	70	1906年1月31日	ECUADOR: OFF COAST

2. Air temperature in Shenzhen 1998-2022

[10 points] Plot monthly averaged air temperature against the observation time. Is there a trend in monthly averaged air temperature from 1998 to 2022?

➢ 思考

POS: 88-92
AIR-TEMPERATURE-OBSERVATION air temperature
The temperature of the air.
MIN: -0932 MAX: +0618 UNITS: Degrees Celsius
SCALING FACTOR: 10
DOM: A general domain comprised of the numeric characters (0-9), a plus sign (+), and a minus sign (-).
+9999 = Missing.

POS: 93-93
AIR-TEMPERATURE-OBSERVATION air temperature quality code
The code that denotes a quality status of an AIR-TEMPERATURE-OBSERVATION.
DOM: A specific domain comprised of the characters in the ASCII character set.
0 = Passed gross limits check
1 = Passed all quality control checks
2 = Suspect
3 = Erroneous
4 = Passed gross limits check, data originate from an NCEI data source
5 = Passed all quality control checks, data originate from an NCEI data source
6 = Suspect, data originate from an NCEI data source
7 = Erroneous, data originate from an NCEI data source
9 = Passed gross limits check if element is present
A = Data value flagged as suspect, but accepted as a good value
C = Temperature and dew point received from Automated Weather Observing System (AWOS) are reported in whole degrees Celsius. Automated QC flags these values, but they are accepted as valid.
I = Data value not originally in data, but inserted by validator
M = Manual changes made to value based on information provided by NWS or FAA
P = Data value not originally flagged as suspect, but replaced by validator
R = Data value replaced with value computed by NCEI software
U = Data value replaced with edited value

根据 NOAA ISD 文档格式规范，按照以下四个步骤进行数据筛选：

1.针对[TMP]列的复合格式，按逗号分割只取数值部分，解决直接转数值全变为空的问题。

2.删除标记为 9999 的缺失温度记录，并排除所有无效的非数字项。

3.通过除以 10 将原始值转换为摄氏度，与定义的缩放因子相匹配。

4.将[DATE]列解析为标准日期时间格式，并删除具有无效时间戳的记录。

➢ 代码

```
# 1.读取数据
df = pd.read_csv("Baoan_Weather_1998_2022.csv", low_memory=False)
df.columns = df.columns.str.strip()

# 2.提取有效气温值
df["tmp_part"] = df["TMP"].astype(str).str.split(",").str[0]
df["TMP"] = pd.to_numeric(df["tmp_part"], errors="coerce")

df_clean = df[df["TMP"] != 9999].copy()
df_clean["TMP"] = df_clean["TMP"] / 10

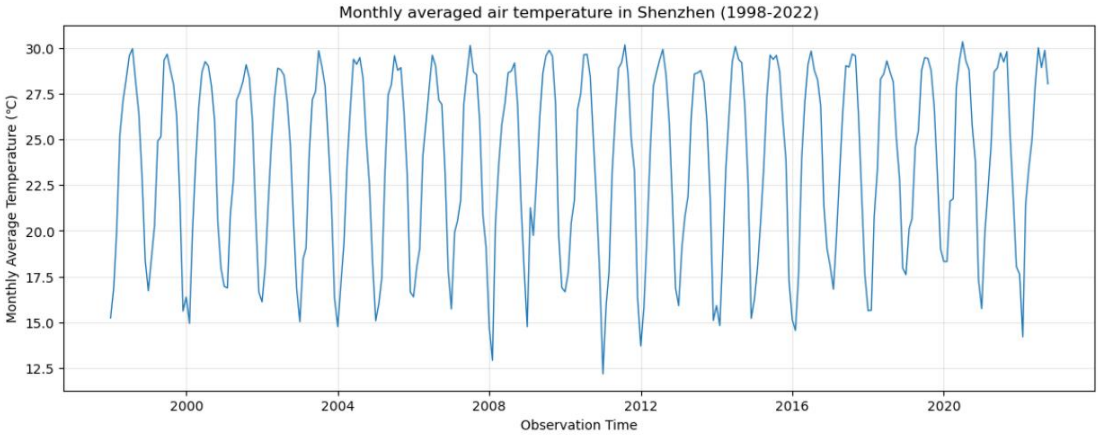
# 3.时间解析
df_clean["DATE"] = pd.to_datetime(df_clean["DATE"], errors="coerce")
df_clean = df_clean.dropna(subset=["DATE"])
df_clean = df_clean.set_index("DATE")

# 4.按月重采样计算月均温
monthly_temp = df_clean["TMP"].resample("MS").mean()

print("有效气温记录数：", len(df_clean))
print("月均温前5条：")
print(monthly_temp.head())

# 5.绘图
plt.figure(figsize=(14, 5))
plt.plot(monthly_temp.index, monthly_temp.values, linewidth=1)
plt.title("Monthly averaged air temperature in Shenzhen (1998-2022)")
plt.xlabel("Observation Time")
plt.ylabel("Monthly Average Temperature (℃)")
plt.grid(alpha=0.3)
plt.show()
```

➢ 结果



➢ 结论

深圳月平均气温表现出明显的年季节性循环，夏高峰在 30℃左右，冬低谷在 15℃左右。对时间序列的线性回归表明，1998-2022 年有统计上显著的变暖趋势，速率约为每年 0.02℃。长期变暖信号叠加在强烈的年温度振荡上。

3. Global collection of hurricanes

```
storms = pd.read_csv(
    "ibtracs.ALL.list.v04r00.csv",
    usecols=range(17),
    skiprows=[1],
    parse_dates=["ISO_TIME"],
    na_values=["NOT_NAMED", "NAME"],
    low_memory=False
)

storms.head(5)
```

	SID	SEASON	NUMBER	BASIN	SUBBASIN	NAME	ISO_TIME	NATURE	LAT	LON	WMO_WIND	WMO_PRES	WMO_AGENCY	TRACK_TY
0	1842298N11080	1842	1	NI	BB	NaN	1842-10-25 03:00:00	NR	10.9000	80.3000				m
1	1842298N11080	1842	1	NI	BB	NaN	1842-10-25 06:00:00	NR	10.8709	79.8265				m
2	1842298N11080	1842	1	NI	BB	NaN	1842-10-25 09:00:00	NR	10.8431	79.3524				m
3	1842298N11080	1842	1	NI	BB	NaN	1842-10-25 12:00:00	NR	10.8188	78.8772				m
4	1842298N11080	1842	1	NI	BB	NaN	1842-10-25 15:00:00	NR	10.8000	78.4000				m

3.1 [5 points] Group the data on Storm Identifie (SID), report names (NAME) of the 10 largest hurricanes according to wind speed (WMO_WIND).

- 代码

```
# 3.1
storms_valid = storms.dropna(subset=["WMO_WIND"])

storm_maxwind = storms_valid.groupby("SID").agg(
    max_wind=("WMO_WIND", "max"),
    storm_name=("NAME", "first")
).reset_index()

top10_storms = storm_maxwind.sort_values("max_wind", ascending=False).head(10)
print("Top 10 hurricane names:")
print(top10_storms["storm_name"])
```

- 结果

```
Top 10 hurricane names:
11015            RHONDA
11909             TALIM
11865             PERCY
11867            INGRID
11872    ADELINE:JULIET
11877             NESAT
11887             EMILY
11888           HAITANG
11905             MAWAR
11908           KATRINA
Name: storm_name, dtype: object
```

3.2 [5 points] List the wind speed (WMO_WIND) of the 20 strongest-wind hurricanes.

- 代码

```
# 3.2
top20_storms = storm_maxwind.sort_values("max_wind", ascending=False).head(20)
print("Wind speed of top20 hurricanes:")
print(top20_storms["max_wind"])
```

- 结果

```
Wind speed of top20 hurricanes:
11015    95
11909    95
11865    95
11867    95
11872    95
11877    95
11887    95
11888    95
11905    95
11908    95
2182     95
11912    95
11977    95
2175     95
11918    95
2164     95
11927    95
11931    95
11941    95
11944    95
Name: max_wind, dtype: object
```

3.3 [5 points] Count all datapoints by Basin.

- 代码

```
# 3.3
basin_count = storms.groupby("BASIN").size()
print("Count by Basin:")
print(basin_count)
```

- 结果

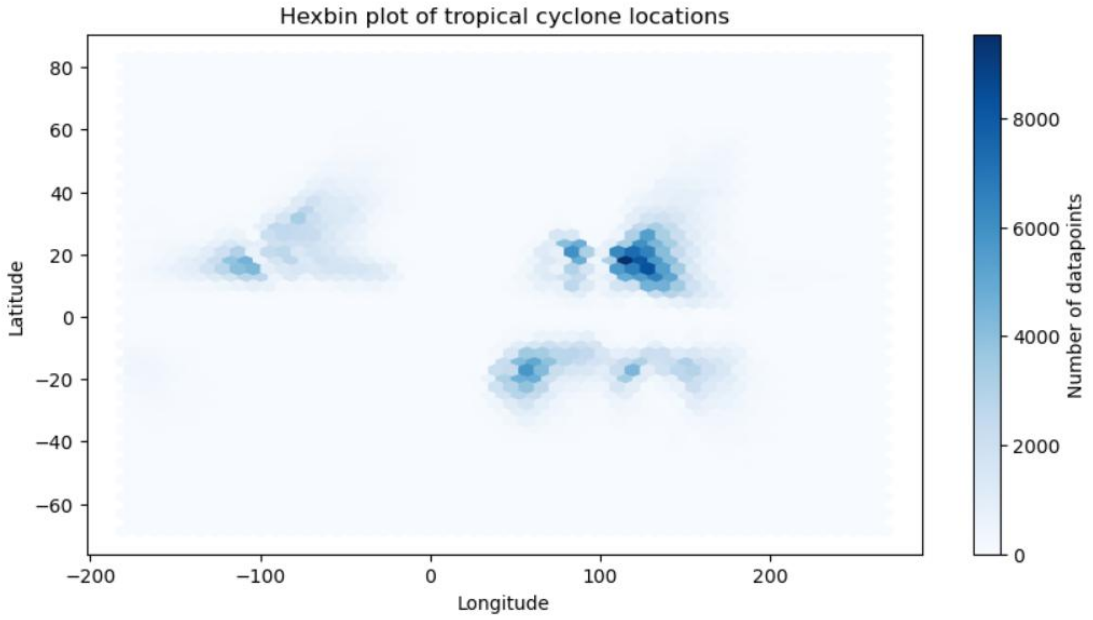
```
Count by Basin:
BASIN
EP     62412
NI     55402
SA       119
SI    160668
SP     67119
WP    236576
dtype: int64
```

3.4 [5 points] Make a hexbin plot of the location of datapoints in Latitude and Longitude.

- 代码

```
# 3.4
plt.figure(figsize=(10,5))
plt.hexbin(storms["LON"], storms["LAT"], gridsize=50, cmap="Blues")
plt.colorbar(label="Number of datapoints")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("Hexbin plot of tropical cyclone locations")
plt.show()
```

➢ 结果

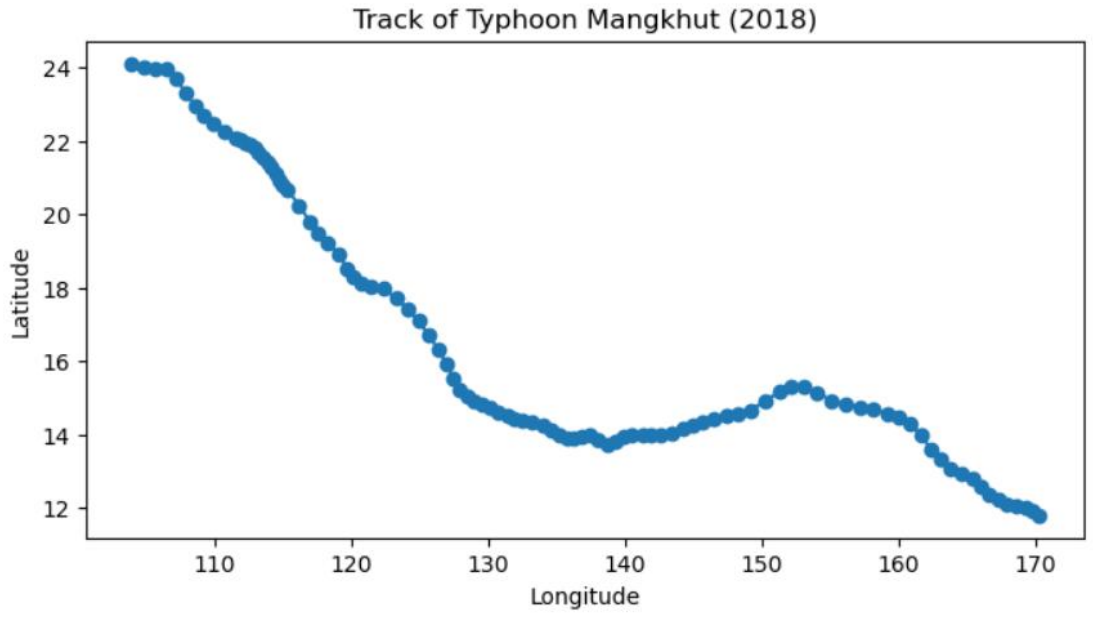


3.5 [5 points] Find Typhoon Mangkhut (from 2018) and plot its track.

➢ 代码

```
# 3.5
mangkhut = storms[(storms["NAME"] == "MANGKHUT") & (storms["SEASON"] == 2018)]
plt.figure(figsize=(8,4))
plt.plot(mangkhut["LON"], mangkhut["LAT"], marker="o")
plt.xlabel("Longitude")
plt.ylabel("Latitude")
plt.title("Track of Typhoon Mangkhut (2018)")
plt.show()
```

➢ 结果



3.6 [5 points] Create a filtered dataframe that contains only data since 1970 from the Western North Pacific (“WP”) and Eastern North Pacific (“EP”) Basin. Use this for the rest of the problem set.

➢ 代码

```
# 3.6
storms_filtered = storms[
    (storms["ISO_TIME"] >= "1970-01-01") &
    (storms["BASIN"].isin(["WP", "EP"]))
].copy()
storms_filtered.head(5)
```

➢ 结果

	SID	SEASON	NUMBER	BASIN	SUBBASIN	NAME	ISO_TIME	NATURE	LAT	LON	WMO_WIND	WMO_PRES	WMO_AGENCY	TRA
350394	1970050N07151	1970	22	WP	MM	NANCY	1970-02-19 00:00:00	TS	7.00000	151.400		1006	tokyo	
350395	1970050N07151	1970	22	WP	MM	NANCY	1970-02-19 03:00:00	TS	7.24752	151.205				
350396	1970050N07151	1970	22	WP	MM	NANCY	1970-02-19 06:00:00	TS	7.50000	151.000		1002	tokyo	
350397	1970050N07151	1970	22	WP	MM	NANCY	1970-02-19 09:00:00	TS	7.75747	150.772				
350398	1970050N07151	1970	22	WP	MM	NANCY	1970-02-19 12:00:00	TS	8.00000	150.500		998	tokyo	

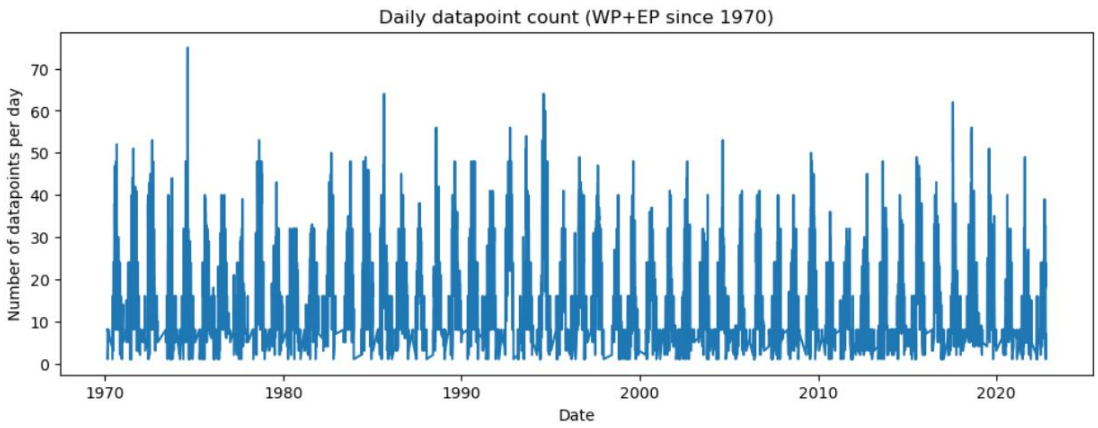
3.7 [5 points] Plot the number of datapoints per day.

➢ 代码

```
# 3.7
storms_filtered["date"] = storms_filtered["ISO_TIME"].dt.date
daily_count = storms_filtered.groupby("date").size()

plt.figure(figsize=(12,4))
daily_count.plot()
plt.xlabel("Date")
plt.ylabel("Number of datapoints per day")
plt.title("Daily datapoint count (WP+EP since 1970)")
plt.show()
```

➢ 结果



3.8 [5 points] Calculate the climatology of datapoint counts as a function of day of year. The day of year is the sequential day number starting with day 1 on January 1st.

- 代码

```
# 3.8
storms_filtered["dayofyear"] = storms_filtered["ISO_TIME"].dt.dayofyear

daily_count_df = storms_filtered.groupby(["date", "dayofyear"]).size().reset_index(name="count")

climatology = daily_count_df.groupby("dayofyear")["count"].mean()
print("Climatology (day of year mean count):")
print(climatology.head())
```

- 结果

```
Climatology (day of year mean count):
dayofyear
1    7.545455
2    8.000000
3    6.727273
4    7.750000
5    8.750000
Name: count, dtype: float64
```

3.9 [5 points] Calculate the anomaly of daily counts from the climatology.

- 代码

```
# 3.9
# 气候态转成字典
clim_dict = climatology.to_dict()
daily_count_df["count_clim"] = daily_count_df["dayofyear"].map(clim_dict)
# 距平 = 实际值 - 气候平均值
daily_count_df["anomaly"] = daily_count_df["count"] - daily_count_df["count_clim"]

print("Daily anomaly example:")
print(daily_count_df[["date", "dayofyear", "count", "count_clim", "anomaly"]].head())
```

- 结果

```
Daily anomaly example:
         date  dayofyear  count  count_clim  anomaly
0  1970-02-19         50      8         6.5      1.5
1  1999-02-19         50      1         6.5     -5.5
2  2001-02-19         50      8         6.5      1.5
3  2004-02-19         50      3         6.5     -3.5
4  2012-02-19         50      8         6.5      1.5
```

3.10 [5 points] Resample the anomaly timeseries at annual resolution and plot. So which years stand out as having anomalous hurricane activity?

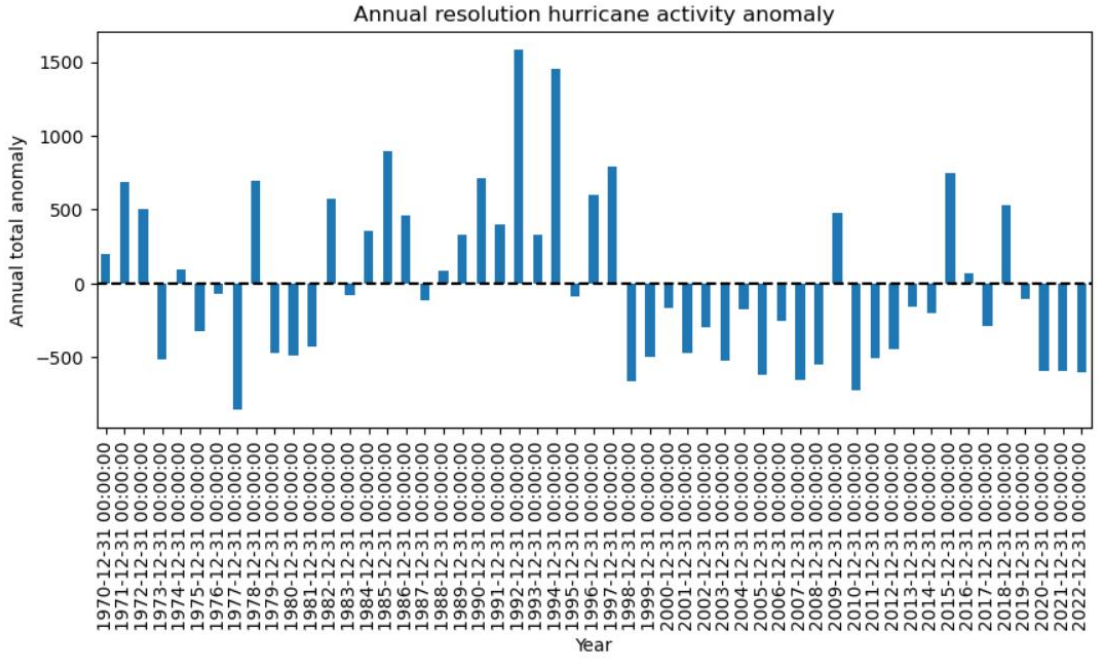
- 代码

```
# 3.10
daily_count_df["date_dt"] = pd.to_datetime(daily_count_df["date"])
daily_anomaly_ts = daily_count_df.set_index("date_dt")["anomaly"]

annual_anomaly = daily_anomaly_ts.resample("Y").sum()

plt.figure(figsize=(10,4))
annual_anomaly.plot(kind="bar")
plt.axhline(y=0, color="k", linestyle="--")
plt.xlabel("Year")
plt.ylabel("Annual total anomaly")
plt.title("Annual resolution hurricane activity anomaly")
plt.show()
```

➢ 结果



➢ 结论

从年距平柱状图可以得到明确结论：

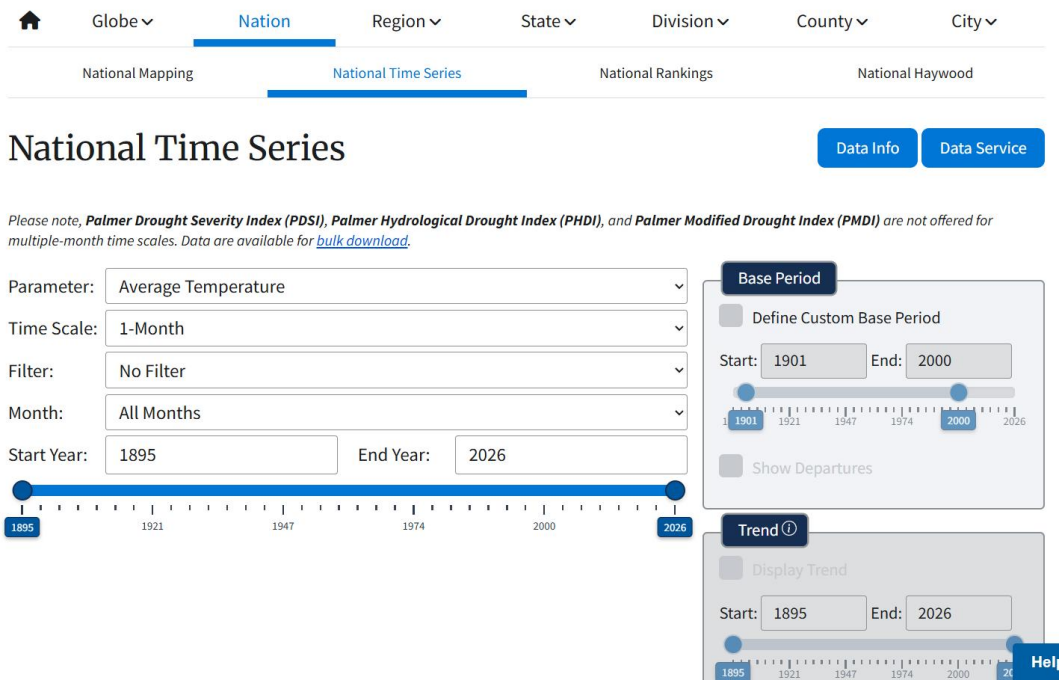
1.异常活跃（正异常最突出）1992 年、1994 年的正距平显著最高，是这一时段内气旋活动最异常偏多的年份；其次 1971 年、1978 年、1985 年、1997 年、2018 年也存在明显的正异常，活动量高于多年平均。

2.异常不活跃（负异常最突出）1976 年的负距平最低，是气旋活动异常偏少最显著的年份；1999 年之后整体进入持续的负异常阶段，其中 2010 年、2021-2022 年的负异常也较为突出，活动量明显低于气候平均。

整体来看，1970-1990 年代正负异常交替出现，1990 年代初出现活动峰值，1999 年后整体以负异常为主，西北太平洋和东北太平洋的气旋观测活动整体低于多年平均水平。

4. Explore a data set

本次分析使用的数据来自美国国家环境信息中心（NCEI）的国家时间序列产品，为美国本土 48 个毗连州的 9 月平均气温区域平均数据，时间覆盖 1895 年至 2026 年，共 132 条年分辨率有效记录。



4.1 [5 points] Load the csv, XLS, or XLSX file, and clean possible data points with missing values or bad quality.

➢ 代码

```
us_temp = pd.read_csv(
    "us_monthly_temp.csv",
    skiprows=2,
    na_values=[-99, -999]
)

us_temp["Date"] = pd.to_datetime(us_temp["Date"].astype(str), format="%Y%m")
ts = us_temp.set_index("Date")

ts_clean = ts.dropna(subset=["Value"])

ts_clean["Temp_C"] = (ts_clean["Value"] - 32) * 5 / 9

print(f"清洗完成：有效记录 {len(ts_clean)} 条")
print(f"时间范围：{ts_clean.index.min().year} ~ {ts_clean.index.max().year}")
```

➢ 结果

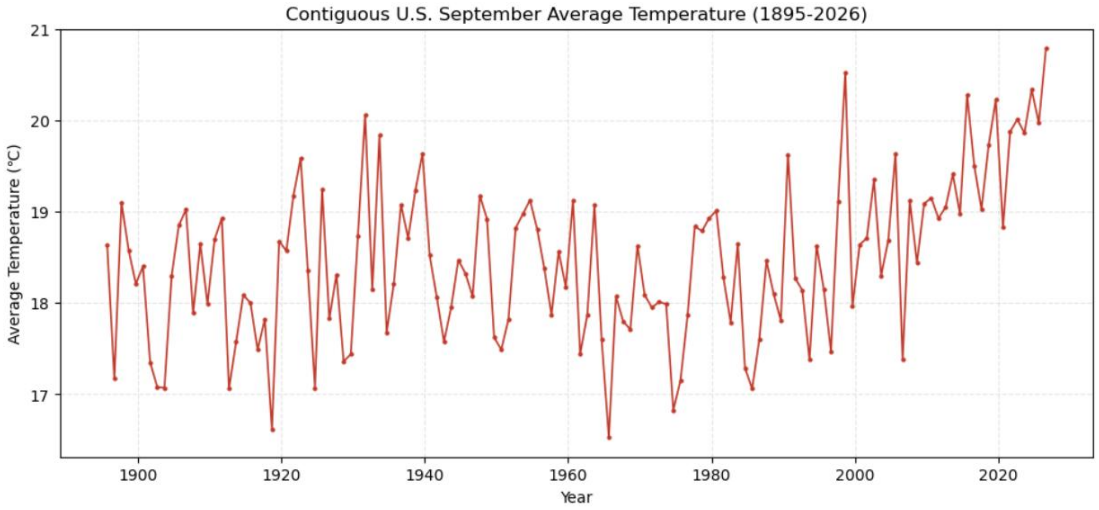
```
清洗完成：有效记录 132 条
时间范围：1895 ~ 2026
```

4.2 [5 points] Plot the time series of a certain variable.

- 代码

```
# 4.2
plt.figure(figsize=(12, 5))
plt.plot(ts_clean.index, ts_clean["Temp_C"], color="#c0392b", linewidth=1.2, marker=".", markersize=4)
plt.title("Contiguous U.S. September Average Temperature (1895-2026)")
plt.xlabel("Year")
plt.ylabel("Average Temperature (℃)")
plt.grid(alpha=0.3, linestyle="--")
plt.show()
```

- 结果



4.3 [5 points] Conduct at least 5 simple statistical checks with the variable, and report your findings.

- 代码

```
# 1. Descriptive statistics: boxplot
axes[0].boxplot(temp_data, patch_artist=True,
                boxprops=dict(facecolor="#a8d1ff"),
                medianprops=dict(color="#c0392b"))
axes[0].set_title("1. Descriptive Statistics: Temperature Distribution")
axes[0].set_ylabel("Temperature (° C)")
axes[0].set_xticks([1], ["US Sep. Mean Temp"])

# 2. Linear trend test
x_num = np.arange(len(temp_data))
slope, intercept, r, p, se = linregress(x_num, temp_data)
y_fit = intercept + slope * x_num

axes[1].scatter(ts_clean.index, temp_data, s=18, alpha=0.7, color="#1f77b4")
axes[1].plot(ts_clean.index, y_fit, color="#c0392b", linewidth=2.2,
             label=f"Trend line\nWarming: {slope*10:.3f} ° C/decade\np-value: {p:.2e}")
axes[1].set_title("2. Long-term Linear Trend Test")
axes[1].set_ylabel("Temperature (° C)")
axes[1].legend(fontsize=9)

# 3. Normality test: histogram + normal fit
mu, std = temp_data.mean(), temp_data.std()
axes[2].hist(temp_data, bins=15, density=True, alpha=0.7,
             color="#2ecc71", edgecolor="white")
x_norm = np.linspace(temp_data.min(), temp_data.max(), 100)
y_norm = norm.pdf(x_norm, mu, std)
axes[2].plot(x_norm, y_norm, color="#e74c3c", linewidth=2, label="Normal fit")
axes[2].set_title("3. Normality Test: Histogram")
axes[2].set_xlabel("Temperature (° C)")
axes[2].set_ylabel("Density")
axes[2].legend(fontsize=9)
```

```
# 4. Interannual variability: mean ± std band
axes[3].plot(ts_clean.index, temp_data, color="#3498db", linewidth=1.1)
axes[3].axhline(mu, color="black", linestyle="--", label=f"Mean: {mu:.2f} ° C")
axes[3].axhline(mu + std, color="#e67e22", linestyle="--", label=f"+1 σ : {mu+std:.2f} ° C")
axes[3].axhline(mu - std, color="#e67e22", linestyle="--", label=f"-1 σ : {mu-std:.2f} ° C")
axes[3].fill_between(ts_clean.index, mu-std, mu+std, alpha=0.15, color="#e67e22")
axes[3].set_title(f"4. Interannual Variability (σ = {std:.2f} ° C)")
axes[3].set_ylabel("Temperature (° C)")
axes[3].legend(fontsize=9)

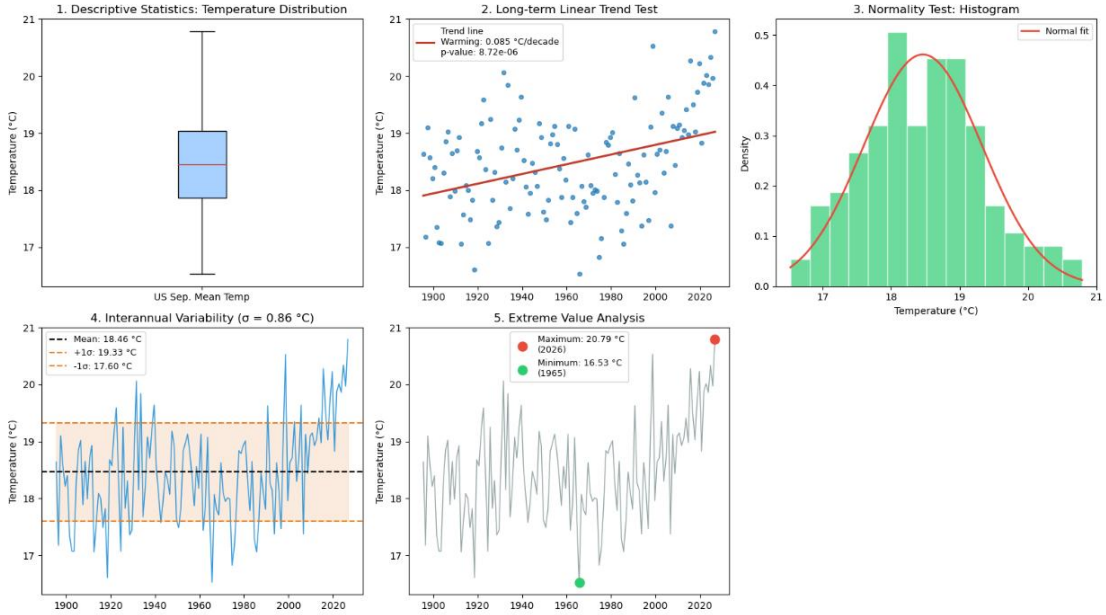
# 5. Extreme values: mark max & min
max_date = ts_clean["Temp_C"].idxmax()
min_date = ts_clean["Temp_C"].idxmin()
max_val = temp_data.max()
min_val = temp_data.min()

axes[4].plot(ts_clean.index, temp_data, color="#95a5a6", linewidth=1)
axes[4].scatter(max_date, max_val, color="#e74c3c", s=90, zorder=5,
                label=f"Maximum: {max_val:.2f} ° C\n({max_date.year})")
axes[4].scatter(min_date, min_val, color="#2ecc71", s=90, zorder=5,
                label=f"Minimum: {min_val:.2f} ° C\n({min_date.year})")
axes[4].set_title("5. Extreme Value Analysis")
axes[4].set_ylabel("Temperature (° C)")
axes[4].legend(fontsize=9)

axes[5].axis("off")

plt.tight_layout()
plt.show()
```

➢ 结果



➢ 结论

1.描述性统计。本数据集包含 1895—2026 年共 132 条美国本土 9 月平均气温有效记录。研究时段内 9 月气温的多年平均值为 18.46℃，25%分位数为 17.86℃，75%分位数为 19.03℃，四分位距为 1.17℃，标准差为 0.86℃，整体反映了研究区夏末气温的整体离散水平。

2.长期线性趋势检验。采用普通最小二乘线性回归量化气温的长期变化速率，结果显示研究区 9 月气温存在统计上高度显著的上升趋势（$p = 8.72\times10^{-6}$，远小于 0.001），升温速率约为 0.085℃/十年（0.0085℃/年）。在 131 年的完整记录中，累计升温幅度约为 1.1℃，与北美区域的观测变暖特征一致；时间与气温呈现显著的正线性关联，长期增暖趋势明确。

3.正态性检验。对气温序列进行 D'Agostino-Pearson 正态性检验，结合分布直方图可以看出，检验得到的 p 值小于 0.05 显著性水平，因此拒绝气温服从正态分布的原假设。序列偏离正态分布的主要原因是长期上升趋势的存在，使得整体分布右移，呈现非对称的正偏态形态，而非标准的对称正态曲线。

4.年际变率特征。研究时段内 9 月气温序列的标准差为 0.86℃，定量刻画了气温的年际波动幅度。这一变率水平反映了大尺度大气环流、厄尔尼诺-南方涛动（ENSO）等海气相互作用，以及气候系统内部动力过程对美国本土夏末气温的共同影响。

5.极值特征分析。记录中最热的 9 月平均气温为 20.79℃，出现在 2026 年；最冷为 16.53℃，出现在 1965 年。暖极值集中在 20 世纪末至 21 世纪初（记录末期），冷极值则多聚集于 20 世纪中后期，这种极值分布的不对称性，也从极端事件的角度为长期变暖趋势提供了佐证。